

Indian Institute of Technology Patna
Department of Computer Science and Engineering
CS3103 – Machine Learning | Laboratory Session 1

Smart Campus Hostel Mess Management System

A command-line tool for meals, attendance, complaints and monthly bills

Submitted by: Rahul Kumar Sahoo
Roll number: 2401CS09
Programme: B.Tech (CSE), 3rd Year
Date: 28 July 2026

1 Title

Smart Campus Hostel Mess Management System – a Python program that keeps mess data in CSV files and sends notifications over Gmail SMTP.

2 Aim

The lab sheet asks us to take a real office job, running a hostel mess, and turn it into a working command-line program. There were five things it needed to do properly: keep a list of students, show and edit the weekly menu, mark daily attendance, log and resolve complaints, and print each student's monthly bill. All the data has to survive a restart, so it goes into CSV files, and the program should not crash on bad input such as a wrong date or a student who has no attendance recorded yet.

3 Platform

I wrote and ran the program on plain Python 3, and also checked that it runs in Google Colab after mounting Drive. Nothing outside the standard library was needed.

Item	What I used
Language	Python 3.x
Where it runs	VS Code / a terminal, and Google Colab
Data storage	CSV files through the csv module
E-mail	Gmail SMTP (STARTTLS, port 587) with a Gmail App Password
Modules	csv, os, pathlib, smtplib, datetime, dataclasses, email.mime, getpass

4 Introduction

A hostel mess office spends most of its time on the same handful of jobs: adding new residents, ticking off who ate which meal, noting down complaints, and totalling the month's bill. Done on paper these things go wrong in small, annoying ways – the same roll number gets written twice, the attendance sheet only lists who was present so nobody can tell who missed the meal, and the final bill picks up an odd arithmetic slip somewhere.

This program moves all of that onto the computer without making it complicated. There is no database and no web page. It uses lists, dictionaries, a couple of small classes and CSV files, which is the toolset this course has covered so far. A menu-driven loop lets a mess committee member pick what they want to do, and each action reads from or writes to the right CSV file, so the next run picks up exactly where the last one left off.

5 Methodology

When the program starts it asks once for a Gmail address and an App Password (pressing Enter twice skips this and the program runs without e-mail), and then it checks that the three CSV files exist, creating them with the correct header row if they do not. After that it drops into the main menu.

Students are stored as a small Student dataclass and written to students.csv. Before adding anyone the roll number is looked up first so a duplicate is never written. The weekly menu is a nested dictionary of the form {day: {meal: item}}, which made both “show the whole week” and “change Tuesday's lunch” easy to write.

Attendance was the part I spent the most time on. The committee member types the roll numbers that were present for a meal. The program checks the date and meal type, saves those students as *present*, and then walks the master student list and writes everyone else as *absent*. If a sheet for that date and meal

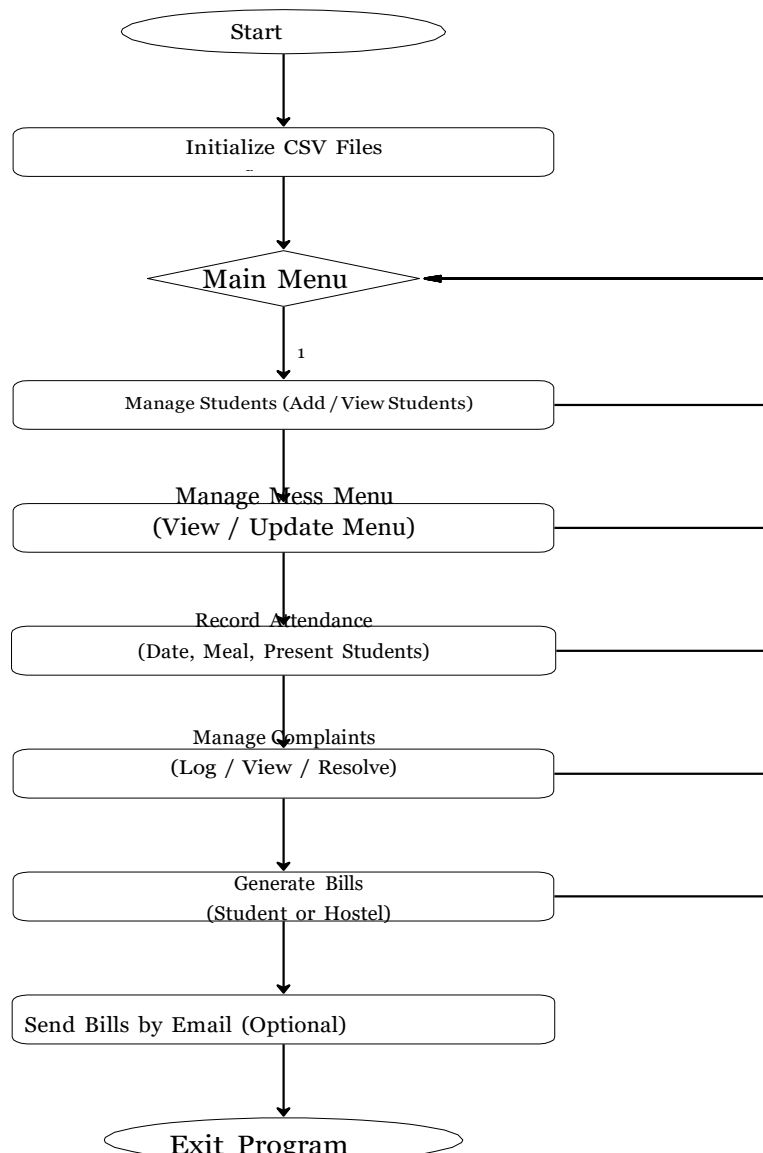
already exists, it raises a `DuplicateAttendanceError` and asks whether to overwrite or keep the old one, instead of silently doubling the rows.

Complaints get a running ID like CP-0001, are saved with status Open, and trigger a confirmation e-mail to the student. Resolving one flips the status to Resolved and sends a second e-mail with the remarks. The bill module counts the *present* rows for a chosen month and applies a fixed base charge plus a per-meal rate. A single student's bill can be printed and mailed, and a hostel-wide run writes every bill into `hostel_bills_<month>.csv`.

Data Design

Entity / File	Main fields	What it is for
Student / <code>students.csv</code>	roll number, name, mail id, hostel	The master list of residents.
Attendance / <code>attendance.csv</code>	roll number, date, meal type, status	One row per student per meal (present / absent).
Complaint / <code>complaints.csv</code>	complaint id, roll number, date, text, status	Complaints and whether they are resolved.
Bill / <code>hostel_bills_<month>.csv</code>	roll, name, meals, charges, total	The generated hostel bill summary.

6 Flow Chart



7 Algorithm

A. Recording attendance

1. Read the date, the meal type and the list of present roll numbers.
2. Check the date is in YYYY-MM-DD form and the meal is Breakfast, Lunch or Dinner.
3. If a sheet for that date and meal already exists, raise `DuplicateAttendanceError` and ask to overwrite or cancel.
4. Check every entered roll number against `students.csv`; reject unknown ones.
5. Write the present students, then loop the master list and write the rest as absent.
6. Go back to the menu.

B. Handling a complaint

1. Confirm the roll number exists.
2. Make the next ID in the CP-0001 series.
3. Save the complaint with status Open and e-mail the student.
4. To resolve, find the ID, set status to Resolved, and e-mail the remarks.

C. Generating a bill

1. Look up the student by roll number.
2. Count that student's present rows for the chosen month.
3. If there are none, treat the count as zero instead of failing.
4. Work out the total with the formula below and show / mail the bill.

$$\text{Total} = \text{Rs. } 2500 + (\text{meals consumed} \times \text{Rs. } 65)$$

8 Code Link

The program is submitted separately as `2401CS09_CS3103_Lab1.py`. It can also be run directly from the Colab notebook below:

<https://colab.research.google.com/drive/137KkTPQ7SUOUrqhGRTw1tecFXHLR15lp?usp=sharing>

9 Supporting File Link

The Drive folder below has the generated `students.csv`, `attendance.csv`, `complaints.csv` and a sample hostel bill file:

https://drive.google.com/drive/folders/1VTt2RO_B3Kq0xBdjE1fnPNcV6nYjjLt7?usp=drive_link

10 Result and Discussion

I ran the program from start to finish and tried the cases most likely to break it. The table lists what was tested and what happened.

The five modules talk to each other cleanly through the one menu, and the two things I was most worried about held up. Attendance never leaves a half-filled sheet because the absentees get filled in automatically, and the bill still comes out fine for a student who has no meals recorded.

What I tested	What happened	Result
Add a new student	Row appended to students.csv.	Pass
Same roll again	Rejected, old data untouched.	Pass
Show weekly menu	Printed straight from the nested dict.	Pass
Attendance twice	Asked me to overwrite or cancel.	Pass
Absentees	Everyone not marked present became absent.	Pass
Log a complaint	Got CP-0001 and status Open.	Pass
Bill, no attendance	Base-only bill, no crash.	Pass
Sample sum (3 meals)	$2500 + 3 \times 65 = \text{Rs. } 2695$.	Pass
No SMTP set	Warned once, CSV writes kept working.	Pass

The one real bug I hit was in the bill code. My first version assumed there would always be at least one attendance row for the month, so a brand-new student with nothing recorded threw an error instead of getting a bill. I fixed it by catching that case, setting the meal count to zero, and still returning a valid base-charge bill (marked with a BUGFIX comment in the code). I also left the e-mail step non-blocking on purpose: if the App Password is missing or the network is down, it prints a warning and carries on rather than losing the data it was about to save. This is also why the e-mail does not go through on Colab or the Moodle runner, since those sandboxes block outgoing SMTP; on a normal machine it sends fine.

11 Conclusion

By the end I had a single Python program that does the whole job: students, menu, attendance, complaints and bills, all driven from one menu and backed by CSV files. It uses most of what the course has covered so far – namely functions, lists and dictionaries, a couple of dataclasses, file handling and exception handling – with the optional SMTP e-mail on top. If I were to take it further I would move the CSV files into a proper database, add a login so only the committee can edit data, and eventually put a simple interface on top.

12 References

1. Python Software Foundation, *Python 3 Documentation*. <https://docs.python.org/3/>
2. Python Software Foundation, *csv — CSV File Reading and Writing*. <https://docs.python.org/3/library/csv.html>
3. Python Software Foundation, *smtplib — SMTP protocol client*. <https://docs.python.org/3/library/smtplib.html>
4. Google Account Help, *Sign in with App Passwords*. <https://support.google.com/accounts/>